

SoC RISC-V en FPGA con FemtoRV32

Verificación del Canal UART mediante Programa de Eco ASCII

Universidad Nacional de Colombia

Programa: Ingeniería Electrónica

Asignatura: Arquitectura de Computadores

FPGA: Tang Primer 20K — GW2A-LV18PG256C8/I7

Procesador: FemtoRV32 Quark (RV32I)

Herramienta: Gowin EDA 1.9.12 | Toolchain: xpack-riscv-none-elf-gcc-15.2.0

Mayo 2026

Resumen

Este documento describe el diseño, implementación y verificación de un programa de eco ASCII ejecutado sobre el SoC FemtoUN. El programa constituye la etapa de validación del subsistema de comunicación UART previa a la implementación de la calculadora bare-metal: confirma la integridad bidireccional del canal serial, el correcto funcionamiento del mapa de registros del periférico, y establece las condiciones operativas que el firmware de mayor complejidad presupone. Se documentan cuatro hallazgos críticos identificados durante la depuración, cada uno con causa raíz, síntoma observable y corrección aplicada.

Repositorio: `femtorv2-soc-fpga` | Licencia MIT | Verilog + C bare-metal

0 Índice

1. Contexto y Objetivo de la Verificación	2
1.1. Posición en el flujo de desarrollo	2
1.2. Objetivos específicos	2
2. Diseño del Firmware de Eco	2
2.1. Principio de operación	2
2.2. Acceso al periférico UART desde C	2
2.3. Protocolo de transmisión de un byte	3
2.4. Protocolo de recepción de un byte	3
2.5. Impresión del valor decimal sin operadores nativos	3
2.6. Lazo principal	4
3. Mapa de Registros del Periférico UART	4
4. Parámetros del Sistema	5
5. Hallazgos Durante la Depuración	5
5.1. FIX 1 — Configuración de codificación en la terminal	5
5.2. FIX 2 — Declaración de punteros UART como variables globales	6
5.3. FIX 3 — Doble procesamiento del mismo byte recibido	7
5.4. FIX 4 — Tamaño insuficiente de la BRAM de programa	7
6. Resultados y Verificación Funcional	8
6.1. Evidencia de operación correcta	8
6.2. Análisis de la salida	9
6.3. Comportamiento ante el retorno de carro	10
6.4. Resumen de pruebas	10
7. Conclusiones	10
A. Resumen de Correcciones Aplicadas	11
B. Comando de Compilación Utilizado	11

1 Contexto y Objetivo de la Verificación

1.1 Posición en el flujo de desarrollo

El SoC FemtoUN, descrito en detalle en el informe de arquitectura general del proyecto, integra el procesador FemtoRV32 Quark, una memoria BRAM de 8192 palabras de 32 bits y un periférico UART mapeado en el espacio de direcciones de la CPU. Una vez confirmado el correcto funcionamiento del sistema en su dimensión de ejecución de instrucciones, el siguiente punto crítico a validar es la cadena completa de comunicación: desde que el firmware escribe en el registro de datos del UART hasta que el carácter aparece en la terminal del usuario, y en sentido inverso, desde que el usuario pulsa una tecla hasta que el byte queda disponible para lectura en el registro de recepción.

El programa de eco ASCII cumple exactamente esta función: recibe cada byte transmitido desde la terminal y lo devuelve de inmediato, acompañado del valor decimal de su código ASCII. Este comportamiento simple es suficiente para ejercitar simultáneamente la lógica de transmisión, la lógica de recepción, el manejo de las banderas de estado del periférico y la sincronización entre el firmware y el hardware, sin la carga adicional de lógica de aplicación que introduciría cualquier programa más complejo.

1.2 Objetivos específicos

1. Verificar la transmisión correcta de caracteres ASCII desde el firmware hacia la terminal PuTTY a través del periférico UART.
2. Verificar la recepción correcta de bytes enviados desde la terminal, incluyendo la lectura del registro de datos y la gestión de la bandera `rx_avail`.
3. Confirmar que el protocolo de acceso a los registros mapeados en memoria del periférico UART opera según la especificación del hardware.
4. Identificar y documentar los problemas encontrados durante la depuración del canal de comunicación, estableciendo así las condiciones de operación correctas para el firmware de la calculadora.

2 Diseño del Firmware de Eco

2.1 Principio de operación

El programa implementa un lazo de eco continuo: espera la llegada de un byte desde la terminal, lo almacena, imprime una línea de la forma `Tecla: X ASCII: N` donde `X` es el carácter recibido y `N` su valor decimal, y retorna al estado de espera. No existe manejo de comandos ni lógica de estado adicional; la sencillez es deliberada, ya que cualquier comportamiento inesperado del sistema puede atribuirse directamente al canal de comunicación y no a la lógica de la aplicación.

2.2 Acceso al periférico UART desde C

El periférico UART expone dos registros en el espacio de direcciones de la CPU. La manera correcta de referenciarlos desde firmware bare-metal es mediante macros de preprocesador, no mediante punteros globales con valor inicial, por razones que se detallan en la sección de hallazgos (véase FIX 2).

```

1 #define uart_data ((volatile int *)0x00400008)
2 #define uart_ctrl ((volatile int *)0x00400010)

```

Listing 1: Definición de acceso a registros UART mediante macros.

El cualificador `volatile` es indispensable: sin él, el compilador puede eliminar el bucle de espera activa al detectar que la variable no es modificada por el propio hilo de ejecución, produciendo firmware que falla en hardware sin mostrar ningún error en compilación.

2.3 Protocolo de transmisión de un byte

La transmisión de un byte requiere tres pasos secuenciales. Primero, verificar que el transmisor no se encuentra procesando un envío anterior consultando el bit 9 del registro de control (`tx_busy`). Segundo, escribir el byte en el registro de datos. Tercero, generar el pulso de escritura activando el bit 0 (`tx_wr`) y desactivándolo en el ciclo siguiente.

```

1 void enviar_byte(char c) {
2     while (*uart_ctrl & (1 << 9)); /* esperar tx_busy = 0 */
3     *uart_data = (int)c;
4     *uart_ctrl = 1;                /* tx_wr = 1: pulso de inicio */
5     *uart_ctrl = 0;                /* tx_wr = 0: fin del pulso */
6 }

```

Listing 2: Función de transmisión de un byte por UART.

2.4 Protocolo de recepción de un byte

La recepción requiere verificar la disponibilidad de datos mediante el bit 8 (`rx_avail`) antes de leer el registro de datos, y confirmar la lectura mediante un pulso en el bit 1 (`rx_ack`). Una espera adicional hasta que `rx_avail` baje es necesaria para evitar que el firmware procese el mismo byte más de una vez (véase FIX 3).

```

1 char recibir_byte(void) {
2     char t;
3     while (!(uart_ctrl & (1 << 8))); /* esperar rx_avail = 1 */
4     t = (char)(uart_data & 0xFF);
5     *uart_ctrl = 2;                /* rx_ack = 1 */
6     *uart_ctrl = 0;                /* rx_ack = 0 */
7     while (*uart_ctrl & (1 << 8)); /* esperar rx_avail = 0 */
8     return t;
9 }

```

Listing 3: Secuencia completa de recepción de un byte.

2.5 Impresión del valor decimal sin operadores nativos

Dado que el procesador implementa únicamente el conjunto base RV32I, sin la extensión de multiplicación y división de hardware, cualquier uso de los operadores `/` y `%` en el código fuente genera llamadas a funciones de biblioteca en tiempo de ejecución (`__divsi3`, `__modsi3`) que no se encuentran disponibles en un entorno compilado con `-nostartfiles` sin el soporte de `libgcc`. Por esta razón, la conversión del valor ASCII a su representación decimal se implementa mediante restas sucesivas para cada potencia de diez.

```

1 void imprimir_numero(int n) {
2     int d; int iniciado = 0;
3     if (n == 0) { enviar_byte('0'); return; }
4     if (n < 0) { enviar_byte('-'); n = -n; }
5
6     d = 0; while (n >= 100) { n -= 100; d++; }
7     if (d || iniciado) { enviar_byte('0' + d); iniciado = 1; }
8     d = 0; while (n >= 10) { n -= 10; d++; }
9     if (d || iniciado) { enviar_byte('0' + d); iniciado = 1; }
10    enviar_byte('0' + n);
11 }

```

Listing 4: Impresión de entero en decimal sin uso de división ni módulo.

Esta implementación funciona correctamente para el rango de valores ASCII (0–127), que es el único rango que el programa de eco necesita imprimir. Para la calculadora, cuyo rango de resultados puede alcanzar valores del orden de 10^9 , se requiere extender la función a nueve potencias de diez, lo que se documenta en el informe correspondiente.

2.6 Lazo principal

```

1 void main(void) {
2     asm volatile("li sp, 0x00002000");
3
4     enviar_texto("Sistema listo. Esperando entrada...\r\n");
5
6     while (1) {
7         char tecla = recibir_byte();
8         enviar_texto("Tecla: ");
9         enviar_byte(tecla);
10        enviar_texto(" ASCII: ");
11        imprimir_numero((int)(unsigned char)tecla);
12        enviar_texto("\r\n");
13    }
14 }

```

Listing 5: Lazo principal del programa de eco ASCII.

La inicialización explícita del puntero de pila mediante `asm volatile("li sp, 0x00002000")` es necesaria porque la compilación con `-nostartfiles` omite el código de arranque que normalmente establece este registro antes de transferir el control a `main`. Sin esta instrucción, las llamadas a funciones que usan la pila producen escrituras en direcciones arbitrarias.

3 Mapa de Registros del Periférico UART

El periférico UART mapeado en memoria expone dos direcciones al procesador. La tabla siguiente resume su función en el contexto del programa de eco.

Cuadro 1: Registros del periférico UART accedidos por el firmware de eco.

Dirección	Nombre	Acceso	Descripción
0x00400008	UART_DATA	R/W	Escritura: byte a transmitir [7:0]. Lectura: último byte recibido [7:0].
0x00400010	UART_CTRL	R/W	Ver tabla 2.

Cuadro 2: Campos de bits del registro 0x00400010.

Bit	Nombre	Dir.	Descripción
0	tx_wr	W	Pulso activo alto: inicia transmisión del byte en UART_DATA.
1	rx_ack	W	Pulso activo alto: confirma lectura y limpia rx_avail.
2	LED	W	Control del LED de usuario.
7	rx_error	R	Error de trama: stop bit recibido como 0.
8	rx_avail	R	Byte recibido disponible para lectura.
9	tx_busy	R	Transmisor ocupado procesando un envío.

4 Parámetros del Sistema

Cuadro 3: Parámetros de hardware y software del entorno de verificación.

Parámetro	Valor
FPGA	Tang Primer 20K — GW2A-LV18PG256C8/I7
Frecuencia de reloj	27 MHz
Baud rate UART	120 000 bps
Duración de trama	$\approx 83.3 \mu s$ (10 bits a 120 kbps)
Ciclos por trama	$27,000,000 \div 120,000 \times 10 = 2250$ ciclos
Memoria BRAM	8192 palabras de 32 bits (32 KiB)
Compilador	xpack-riscv-none-elf-gcc 15.2.0
Flags	-march=rv32i -mabi=ilp32 -Os -nostartfiles
Terminal	PuTTY — COM9, 120000 baud, ISO-8859-1

5 Hallazgos Durante la Depuración

Durante el proceso de validación del canal de comunicación se identificaron cuatro problemas que comprometían la operación correcta del sistema. Cada uno se documenta con su síntoma observable, causa raíz y corrección aplicada.

5.1 FIX 1 — Configuración de codificación en la terminal

Síntoma: Algunos caracteres enviados desde la terminal producían en el receptor dos o más bytes en lugar de uno, con códigos ASCII que no correspondían al carácter pulsado.

La letra **n** con tilde, por ejemplo, generaba una secuencia de dos bytes, y determinados caracteres de puntuación producían valores inesperados.

Causa: PuTTY estaba configurado con codificación UTF-8. En este modo, cualquier carácter fuera del rango ASCII estándar (0–127) se representa como una secuencia multibyte según el estándar Unicode. El periférico UART opera byte a byte sin interpretación de codificación: cada byte recibido se trata como un carácter independiente. La discrepancia entre ambas representaciones produce lecturas incorrectas.

Corrección: Cambiar la configuración de traducción de caracteres en PuTTY a ISO-8859-1 (Latin-1). En esta codificación cada carácter ocupa exactamente un byte y el valor numérico del byte coincide con el esperado por el firmware.

Cuadro 4: Configuración de PuTTY para operación correcta con el firmware de eco.

Parámetro	Valor
Puerto serie	COM9
Velocidad	120000 baud
Bits de datos	8
Paridad	None
Bits de parada	1
Eco local	Force off
Edición local de línea	Force off
Traducción (codificación)	ISO-8859-1

5.2 FIX 2 — Declaración de punteros UART como variables globales

Síntoma: El sistema no producía ninguna salida en la terminal pese a que el procesador ejecutaba instrucciones y el periférico UART respondía correctamente en simulación. Las señales de bus mostraban accesos a direcciones dentro del rango de la BRAM en lugar de las direcciones 0x00400008 y 0x00400010.

Causa: Los punteros al periférico UART estaban declarados como variables globales inicializadas:

```
1 volatile int *uart_data = (volatile int *)0x00400008; /* INCORRECTO
   */
2 volatile int *uart_ctrl = (volatile int *)0x00400010; /* INCORRECTO
   */
```

Listing 6: Declaración incorrecta de punteros UART como variables globales.

Con `-nostartfiles`, el código de arranque que copia la sección `.data` desde la memoria de programa hacia la memoria de datos nunca se ejecuta. Las variables globales inicializadas residen en `.data`, por lo que sus valores iniciales nunca se establecen: los punteros contienen basura y los accesos al UART se dirigen a posiciones arbitrarias de la BRAM.

Corrección: Sustituir las variables globales por macros de preprocesador. Una macro no genera ninguna variable ni ocupa espacio en `.data`; el preprocesador reemplaza cada referencia por la expresión literal antes de la compilación, embebiendo las direcciones directamente como constantes en las instrucciones del programa.

```

1 #define uart_data ((volatile int *)0x00400008) /* CORRECTO */
2 #define uart_ctrl ((volatile int *)0x00400010) /* CORRECTO */

```

Listing 7: Declaración correcta mediante macros de preprocesador.

5.3 FIX 3 — Doble procesamiento del mismo byte recibido

Síntoma: Al pulsar una tecla en la terminal, la salida mostraba el mismo carácter y su valor ASCII impresos dos veces consecutivas, como si el firmware hubiera recibido el mismo byte en dos iteraciones distintas del lazo.

Causa: La bandera `rx_avail` (bit 8 del registro de control) no baja de forma inmediata en el mismo ciclo en que se escribe el pulso `rx_ack`. Existe una latencia de al menos un ciclo de reloj entre la escritura del pulso de confirmación y la actualización de la bandera en la salida del registro. Si el firmware regresa al inicio del lazo y evalúa `rx_avail` antes de que esta latencia expire, encuentra la bandera aún activa y procesa el mismo byte por segunda vez.

Corrección: Agregar una espera activa después del pulso `rx_ack` que mantiene al firmware bloqueado hasta que `rx_avail` confirme su bajada:

```

1 *uart_ctrl = 2; /* rx_ack = 1: confirmar lectura
   */
2 *uart_ctrl = 0; /* rx_ack = 0: fin del pulso
   */
3 while (*uart_ctrl & (1 << 8)); /* esperar rx_avail = 0
   */

```

Listing 8: Corrección del doble procesamiento mediante espera de `rx_avail`.

5.4 FIX 4 — Tamaño insuficiente de la BRAM de programa

Síntoma: La síntesis completaba sin errores pero el sistema no ejecutaba ninguna instrucción al programar la FPGA. El análisis con `riscv-none-elf-size` reveló que el firmware superaba la capacidad definida en `bram_hex.v`.

Causa: La capacidad original de la BRAM, de 256 palabras de 32 bits (1 KiB), resultó insuficiente para alojar el firmware junto con las cadenas de texto del código fuente. El proceso de síntesis no detecta este desbordamiento como error: simplemente descarta las instrucciones que no caben, produciendo un binario truncado que el procesador intenta ejecutar con resultados impredecibles.

Corrección: Ampliar el tamaño de la BRAM de forma incremental, verificando con `riscv-none-elf-size` que el firmware cabe antes de sintetizar.

Cuadro 5: Evolución del tamaño de la BRAM durante el proceso de depuración.

Palabras	Capacidad	Resultado
256	1 KiB	Firmware truncado — sistema sin respuesta
1024	4 KiB	Firmware truncado — sistema sin respuesta
2048	8 KiB	Firmware truncado — sistema sin respuesta
4096	16 KiB	Firmware completo (versión sin <code>start.S</code>)
8192	32 KiB	Configuración actual — margen para la calculadora

6 Resultados y Verificación Funcional

6.1 Evidencia de operación correcta

La figura 1 muestra la salida del programa de eco en la terminal PuTTY. Para cada tecla pulsada por el usuario el sistema imprime el carácter seguido de su código ASCII en decimal. La secuencia capturada incluye dígitos, operadores aritméticos, caracteres alfabéticos y el código de retorno de carro (ASCII 13), cubriendo las categorías de entrada relevantes para la calculadora que se implementará a continuación.

```
Tecla: 9 ASCII: 57
Tecla: + ASCII: 43
Tecla: 6 ASCII: 54
Tecla: = ASCII: 61
Tecla: 4 ASCII: 52
Tecla: 5 ASCII: 53
Tecla: + ASCII: 43
Tecla: 2 ASCII: 50
Tecla: = ASCII: 61
Tecla: 5 ASCII: 53
Tecla: - ASCII: 45
Tecla: 2 ASCII: 50
Tecla: = ASCII: 61
  ASCII: 13
  ASCII: 13
  ASCII: 13
  ASCII: 13
Tecla: 5 ASCII: 53
Tecla: - ASCII: 45
Tecla: 2 ASCII: 50
Tecla: = ASCII: 61
  ASCII: 13
Tecla: c ASCII: 99
Tecla: r ASCII: 114
Tecla: p ASCII: 112
Tecla: + ASCII: 43
Tecla: - ASCII: 45
Tecla: * ASCII: 42
Tecla: = ASCII: 61
  ASCII: 13
Tecla: 8 ASCII: 56
Tecla: + ASCII: 43
Tecla: 2 ASCII: 50
Tecla: = ASCII: 61
Tecla: = ASCII: 61
  ASCII: 13
Tecla: 9 ASCII: 57
Tecla: * ASCII: 42
Tecla: 4 ASCII: 52
Tecla: = ASCII: 61
  ASCII: 13
```

Figura 1: Salida del programa de eco en PuTTY. Cada línea corresponde a una tecla pulsada por el usuario; se muestra el carácter recibido y su código ASCII en decimal.

6.2 Análisis de la salida

La tabla siguiente contrasta una selección de entradas observadas en la captura con los valores esperados según la tabla ASCII estándar, confirmando la correspondencia exacta

en todos los casos.

Cuadro 6: Verificación de correspondencia entre carácter recibido y código ASCII impreso.

Carácter	ASCII esperado	ASCII recibido	Categoría
9	57	57	Dígito decimal
+	43	43	Operador suma
6	54	54	Dígito decimal
=	61	61	Símbolo igual
-	45	45	Operador resta
*	42	42	Operador multiplicación
c	99	99	Comando de borrado
r	114	114	Carácter alfabético
p	112	112	Carácter alfabético
CR	13	13	Retorno de carro

6.3 Comportamiento ante el retorno de carro

La captura muestra varias líneas consecutivas con ASCII: 13, correspondientes a múltiples recepciones del código de retorno de carro. Este comportamiento es esperado: con *Local line editing* en *Force off*, PuTTY transmite cada byte de forma inmediata, por lo que la repetición de tecla del sistema operativo puede generar varios bytes idénticos en sucesión. La respuesta del firmware en cada caso es correcta e idéntica, confirmando la ausencia de efectos de estado residual entre iteraciones del lazo de eco.

6.4 Resumen de pruebas

Cuadro 7: Resultados de las pruebas de verificación funcional del canal UART.

Prueba	Estado	Evidencia
TX: impresión de texto en terminal	PASS	Mensaje de bienvenida recibido
RX: recepción de dígitos	PASS	Códigos 48–57 verificados
RX: recepción de operadores	PASS	Códigos 42, 43, 45, 61 verificados
RX: recepción de caracteres alpha	PASS	Códigos 99, 112, 114 verificados
RX: recepción de retorno de carro	PASS	Código 13, múltiples recepciones
Gestión de <code>rx_avail</code>	PASS	Sin doble procesamiento (FIX 3)
Codificación ISO-8859-1	PASS	Un byte por carácter verificado

7 Conclusiones

1. **El canal UART opera correctamente en ambas direcciones.** La correspondencia exacta entre los caracteres enviados y los códigos ASCII reportados por el firmware confirma la integridad del subsistema de comunicación serial, tanto en transmisión como en recepción.
2. **La gestión correcta de `rx_avail` es un requisito no negociable.** La latencia entre el pulso de confirmación y la bajada efectiva de la bandera obliga a incluir la

espera activa del FIX 3. Omitirla produce doble procesamiento silencioso, un defecto que no genera ningún mensaje de error y que puede manifestarse de formas sutiles en firmware de mayor complejidad.

3. **Las direcciones de hardware deben resolverse en tiempo de compilación.** La sustitución de punteros globales por macros de preprocesador elimina la dependencia de la inicialización de `.data`, inexistente en entornos compilados con `-nostartfiles`. Este principio se aplica a todo firmware adicional sobre este SoC.
4. **El programa de eco establece las condiciones operativas para la calculadora.** Los cuatro hallazgos documentados, junto con sus correcciones, definen un conjunto de prácticas de firmware que el desarrollo de la calculadora puede presuponer como validadas. El canal serial queda caracterizado como un componente confiable sobre el cual construir lógica de aplicación de mayor complejidad.

A Resumen de Correcciones Aplicadas

Cuadro 8: Resumen de los cuatro hallazgos documentados durante la verificación.

FIX	Componente	Problema	Corrección
1	PuTTY	UTF-8 genera secuencias multibyte	Cambiar a ISO-8859-1
2	<code>main.c</code>	Punteros UART en <code>.data</code> sin inicializar	Reemplazar por <code>#define</code>
3	<code>main.c</code>	Doble lectura por latencia de <code>rx_avail</code>	Espera activa tras pulso <code>rx_ack</code>
4	<code>bram_hex.v</code>	BRAM insuficiente para el firmware	Ampliar a 8192 palabras

B Comando de Compilación Utilizado

```
riscv-none-elf-gcc -march=rv32i -mabi=ilp32 -Os -nostartfiles ^
-Wl,-Ttext=0x00000000" -o firmware.elf main.c

riscv-none-elf-objcopy -O verilog --verilog-data-width=4 ^
firmware.elf firmware.hex
```

Listing 9: Secuencia de compilación y generación del archivo hex (PowerShell).